

Formulários e Sessões em PHP 2025/1

FACOM32603 - Desenvolvimento Web I

Prof. Ian R. Cunha
ianresende@ufu.br

Formulários com PHP

- Formulários são a principal ponte de comunicação do cliente (navegador) para o servidor (PHP).
- Quando um formulário é enviado, o navegador empacota os dados e os envia.
- O PHP recebe esses dados em **Superglobais**: variáveis especiais, do tipo **array**, que estão sempre disponíveis em qualquer escopo.

Variáveis pré-definidas (ou superglobais)

- **\$_GET** : array associativo de variáveis passadas para o script atual via método HTTP GET, pela **URL**.
- **\$_POST** : array associativo de variáveis passadas para o script atual via método HTTP POST, pelo **corpo da requisição**.
- **\$_FILES**: Informações sobre arquivos enviados.

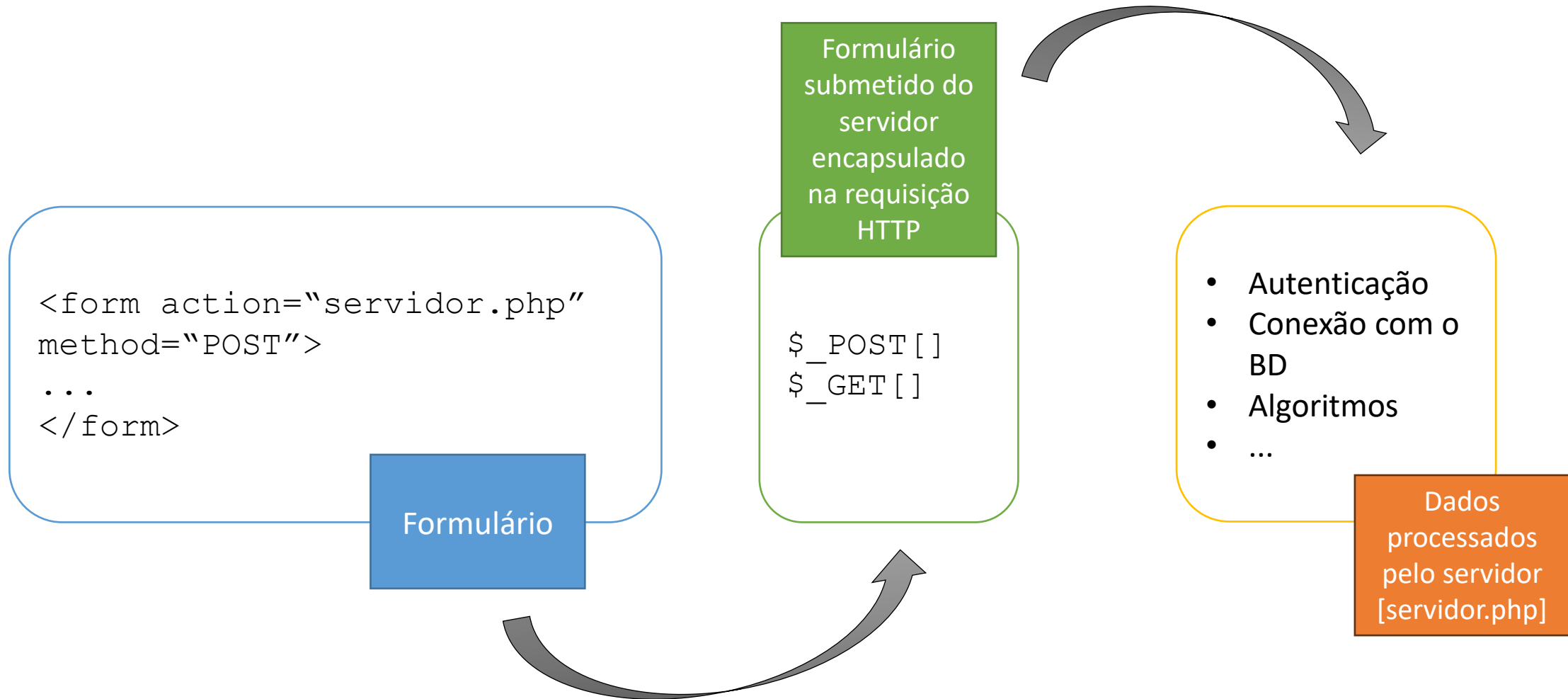
Variáveis pré-definidas (ou superglobais)

- **\$_COOKIE** : array associativo de variáveis passadas para o atual script via HTTP Cookies
- **\$_SESSION** : array associativo contendo variáveis de sessão disponíveis para o atual script
- **\$_REQUEST** : array associativo que reúne o conteúdo de \$_GET, \$_POST e \$_COOKIE

Superglobais de contexto

- **`$GLOBALS`** : um array com referências para todas as variáveis globais do script
- **`$_SERVER`** : um array com informações gerais relacionadas ao servidor
 - Exemplos:
 - `$_SERVER["HTTP_HOST"]`
 - `$_SERVER["SERVER_NAME"]`
 - `$_SERVER["DOCUMENT_ROOT"]`

Envio de formulários para o PHP



Método GET

- Envia os dados do formulário como parte da URL

`http://localhost/cadastra_usuario.php?nome=Fulano&idade=19`

- Caractere **?**
 - Representa o início de uma cadeia de parâmetros
- Símbolo **&**
 - Identifica o início de uma nova variável
- Caractere **=**
 - Separa as variáveis de seus respectivos valores

Método POST

- Envia os dados do formulário no corpo (body) da requisição.
- É o método padrão para enviar dados sensíveis e dados longos
- Dados codificados (enctype): application/x-www-form-urlencoded ou multipart/form-data

application/x-www-form-urlencoded

```
POST /login HTTP/1.1Host: www.exemplo.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 30
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)

username=joao.silva&password=senha123
```

Método POST

multipart/form-data

```
POST /login HTTP/1.1Host: www.exemplo.com
```

```
Content-Type: multipart/form-data
```

```
Content-Length: 30
```

```
User-Agent: Mozilla/5.0
```

```
--ABC_BOUNDARY_12345
```

```
Content-Disposition: form-data; name="nome"
```

```
João
```

```
--ABC_BOUNDARY_12345
```

```
Content-Disposition: form-data; name="imagem";
```

```
filename="foto.jpg" Content-Type: image/jpeg
```

```
[...dados binários da imagem...]
```

```
--ABC_BOUNDARY_12345--
```

GET vs POST

GET	POST
Valores visíveis na URL	Valores não visíveis na URL (envio no corpo da mensagem)
Tem limitação de caracteres, dependendo do navegador (geralmente 255 caracteres)	Não tem limite de caracteres
Mais rápido que POST por causa da simplicidade	Mais lento que o GET por causa do tempo de encapsulamento
Suporta apenas strings	Suporta diferentes tipos de dados (strings, números, binário, etc.)
Fica no histórico do navegador	Não fica no histórico do navegador
Pode ser colocado nos favoritos (bookmark) por causa da URL	Não pode ser colocado nos favoritos (bookmark)

Exemplo: envio de formulário

Documento form.php
Neste caso, poderia ser
.html visto que é um
documento estático.

```
<html>
<head><title>Formulários em PHP</title></head>
<body>
  <h1>Manipulando dados de Formulários em PHP</h1>
  <form action="enviarcontato.php" method="post">
    <label>Nome:</label>
    <input type="text" name="nome"></input>
    <br/><br/>
    <label>Email:</label>
    <input type="text" name="email"></input>
    <br/><br/>
    <label>Descricao:</label>
    <textarea name="descricao" cols="40" rows="5"></textarea>
    <br/><br/>
    <input type="submit" value="Enviar">
  </form>
</body>
</html>
```

Nome do script que
vai tratar o formulário.

Exemplo: envio de formulário

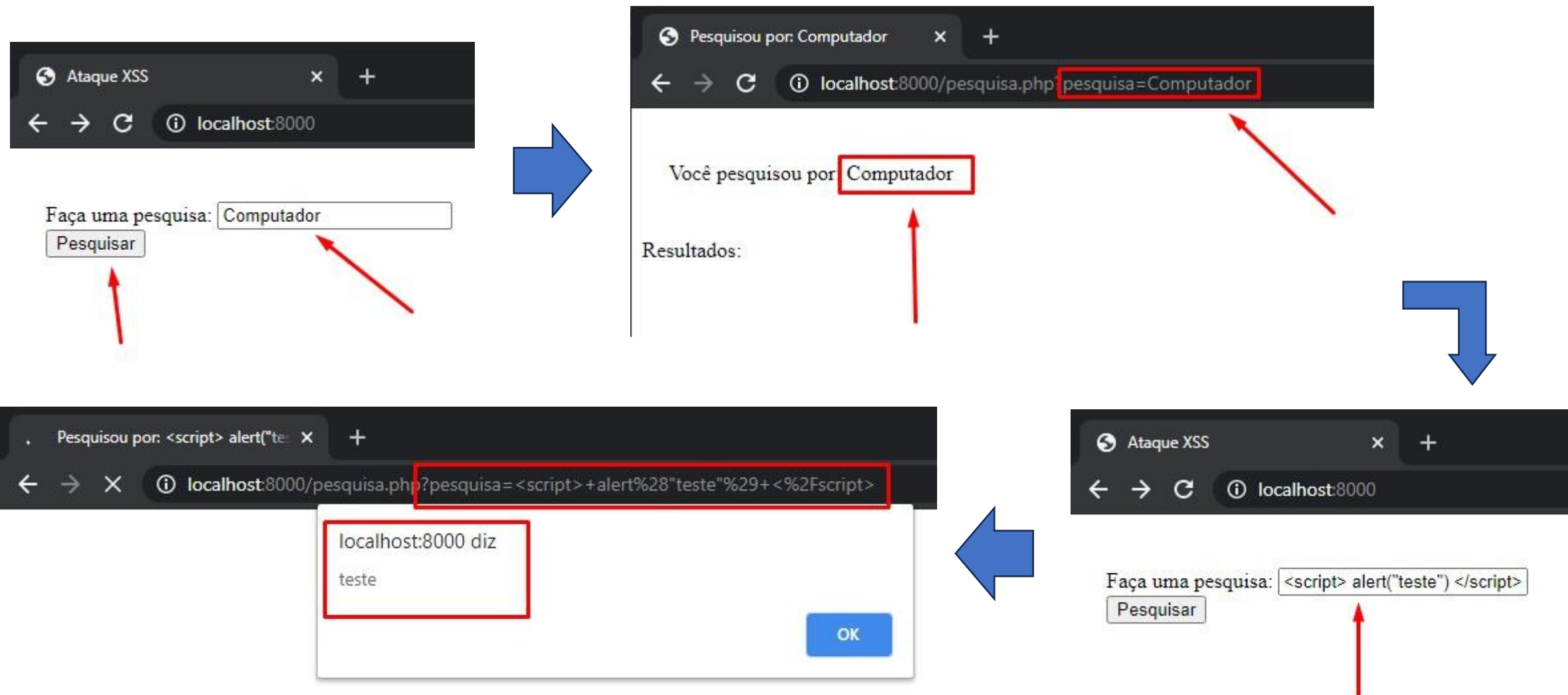
enviarcontato.php

```
<?php
    echo "Sr(a) " . $_POST["nome"] . ", seu contato foi enviado com sucesso!<br/>";
    echo "Texto enviado: " . $_POST["descricao"];
?>
```

Atributo “name” do campo do formulário.

** Conteúdo do script “enviarcontato.php”*

Ataques XSS (*cross-site scripting*)



Ataques XSS (*cross-site scripting*)

- Código malicioso injetado em sites
- Os scripts maliciosos podem acessar cookies, tokens de sessão ou outras informações confidenciais retidas pelo navegador
- O alvo não é o próprio sistema Web, mas os usuários que o utilizam
- Geralmente, os criminosos cibernéticos utilizam os campos de formulários (ou a própria URL) para injetar o código malicioso
- Algumas funções e filtros do PHP auxiliam na prevenção
 - **htmlspecialchars()** : converte caracteres especiais para a entidades HTML, por exemplo,
 - & irá se tornar `&`
 - > irá se tornar `>`
 - < irá se tornar `<`
 - ...

Extensões Filter

- As extensões **Filter** do PHP são utilizadas para recuperar, validar e limpar (**sanitizar**) dados que são recebidos de fontes inseguras, como campos de texto onde o usuário pode digitar qualquer coisa.
- Algumas funções específicas são utilizadas para esse fim, como **filter_input()**, **filter_has_var()**, **filter_input_array()**, **filter_var()** etc.

Recupera uma variável externa pelo nome e opcionalmente a filtra.

Verifica se existe uma variável específica em um dos tipos existentes (INPUT_GET, INPUT_POST, INPUT_COOKIE, INPUT_SERVER, INPUT_ENV).

Recupera um conjunto de variáveis externas e opcionalmente as filtra. Útil para não precisar chamar a função `filter_input()` várias vezes.

Filtros predefinidos

- FILTER_VALIDATE_EMAIL: valida um endereço de e-mail
- FILTER_VALIDATE_INT: valida um inteiro
- FILTER_VALIDATE_URL: Verifica se é uma URL válida.
- FILTER_SANITIZE_SPECIAL_CHARS: remove os caracteres especiais
- FILTER_SANITIZE_URL: remove caracteres ilegais de uma URL
- FILTER_SANITIZE_NUMBER_INT: Remove todos os caracteres exceto dígitos e sinal.

https://www.w3schools.com/php/php_ref_filter.asp

Extensões Filter

- A função `filter_input()` valida e limpa variáveis externas (como campos input de formulários).

```
$search_html = filter_input(INPUT_GET, 'search', FILTER_SANITIZE_SPECIAL_CHARS);
```

Tipo

Nome da
variável

Filtro a ser
aplicado

Retorna o valor da variável requisitada em caso de sucesso, FALSE se o filtro falhar, ou NULL se a variável não existir.

Envio de arquivos

Forma de codificação do conteúdo do corpo da mensagem da requisição.

`multipart/form-data` indica que os dados são enviados por stream em pedaços.

Padrão: `application/x-www-form-urlencoded`

Representação: `campo1=valor1&campo2=valor2&...`

```
<form name="form" method="post" action="enviar_pdf.php"
enctype="multipart/form-data">
    <input type="file" name="txtArquivo" value="" />
    <button type="submit" name="salvar">Salvar</button>
</form>
```

Envio de arquivos

- `$_FILES` : variável global que conterà toda a informação do arquivo enviado/recebido. Por exemplo:
 - `$_FILES['foto']['name']` : nome original do arquivo enviado
 - `$_FILES['foto']['type']`: tipo MIME do arquivo enviado
 - `$_FILES['foto']['size']`: tamanho, em bytes, do arquivo enviado
 - `$_FILES['foto']['error']`: código de erro associado ao envio
- `file_exists` : verifica se um arquivo ou diretório existe
- `mkdir` : tenta criar o diretório especificado
- `move_uploaded_file` : verifica se o arquivo especificado pelo nome é um arquivo de upload válido. Se o arquivo for válido, ele será movido para o nome diretório fornecido pelo destino.

Neste caso, a chave “foto” é o valor do atributo name do input no form.

Exercícios

1. Criar um formulário HTML que possui um elemento *select* para escolher um valor que representa a idade de uma pessoa entre 18 e 50 anos. Criar o *select* dinamicamente com PHP.



18 ▼
18
19
20
21

2. Implementar o envio do formulário via POST para o mesmo script PHP com a inclusão de um elemento `<h1>` com a frase “Você tem XX anos”, substituindo o XX pelo valor escolhido pelo usuário.

Exercícios

3. Criar um formulário HTML que possui um elemento *select* para escolher um valor que representa um estado brasileiro entre “Amazonas”, “Ceará”, “Goiás”, “Minas Gerais” e “Santa Catarina”.
 - Utilize um array associativo com os as respectivas siglas dos estados como chave e o nome por extenso como valor de cada campo.
 - Utilize um *foreach* para criar o *select* dinamicamente.

Exercícios

4. Crie um formulário para executar a função de substituição de textos conforme o exemplo abaixo.

Texto:

Substituir:

Por:



Texto:

Substituir:

Por:

v#c# foi pro brejo

O Problema: A Web é "Stateless" (Sem Estado)

O protocolo HTTP, que rege a web, não tem memória. Cada requisição é independente.

- **Requisição 1:**
 - **Cliente diz:** “Olá, sou o João!” -> **Servidor responde:** “Olá, João!”
- **Requisição 2 (1 segundo depois):**
 - **Cliente diz:** “Me dê acesso ao meu perfil.” -> **Servidor responde:** “**Quem é você?**”

O servidor não tem como saber que a segunda requisição veio da mesma pessoa que a primeira.

A Solução: Autenticação, Sessões e Cookies

- **Autenticação:** É a **portaria do evento**. Você apresenta seu documento (usuário/senha) para provar quem você é.
- **Sessão:** É o **registro na lista de convidados**. A administração do evento marca que você está lá dentro. Essa lista fica com a organização (o **servidor**).
- **Cookie:** É o **crachá** que você recebe na portaria. Ele tem um código único. Você o carrega consigo (no **navegador**).

Para entrar em uma área VIP, você não mostra o documento de novo, apenas o crachá. O segurança confere o código do crachá na lista e te deixa passar.

Sessão

- Sessões podem ser usadas para persistir informações entre requisições de páginas
 - Armazenar dados em memória de um usuário específico
- Array global `$_SESSION`
- A função `session_start()` é usada para iniciar uma sessão manualmente.

Passo 1 - A Autenticação (Verificando a Identidade)

- O processo começa com um formulário de login (method="POST") e um script PHP que valida as credenciais.

```
<?php
$email_form = $_POST['email'];
$senha_form = $_POST['senha'];

// Hash da senha armazenado no banco de dados
$senha_hash_db = '$2y$10$K...';

if (password_verify($senha_form, $senha_hash_db)) {
    // Senha correta! Autenticação bem-sucedida.
    // AGORA, VAMOS INICIAR A SESSÃO...
} else {
    // Senha incorreta.
    echo "Login ou senha inválidos.";
}
?>
```

Passo 2 - Iniciando a Sessão (Após o Login)

- Após a autenticação bem-sucedida, iniciamos a sessão para que o servidor consiga se "lembrar" do usuário.
- Ao iniciar a sessão, o PHP automaticamente envia um cookie com o ID da sessão para o navegador do usuário.

```
if (password_verify($senha_form, $senha_hash_db)) {  
    // 1. Inicia a sessão (ou continua uma existente)  
    session_start();  
  
    // 2. Armazena os dados do usuário na superglobal $_SESSION  
    $_SESSION['logado'] = true;  
    $_SESSION['id_usuario'] = $usuario_db['id'];  
    $_SESSION['nome_usuario'] = $usuario_db['nome'];  
  
    // 3. Redireciona para a área restrita  
    header('Location: painel.php');  
    exit();  
}
```

Passo 3: Protegendo Páginas Restritas

- Em toda página que precisa de login, adicionamos um "bloco de segurança/verificação".

```
<?php
// SEMPRE inicie a sessão no topo de qualquer página que a utilize.
session_start();

// Bloco de segurança: verifica se a sessão NÃO existe ou NÃO está logada.
if (!isset($_SESSION['logado']) || $_SESSION['logado'] !== true) {
    // Se não estiver logado, destrói qualquer sessão iniciada e redireciona
    session_destroy();
    header('Location: login.html');
    exit();
}

// Se o script continuar, o usuário está logado.
$nome = htmlspecialchars($_SESSION['nome_usuario']);
?>
<h1>Bem-vindo ao Painel, <?php echo $nome; ?>!</h1>
```

Passo 4: O Logout (Encerrando a Sessão)

- O logout é um processo de 3 passos para garantir que a sessão seja completamente encerrada no servidor e no cliente.

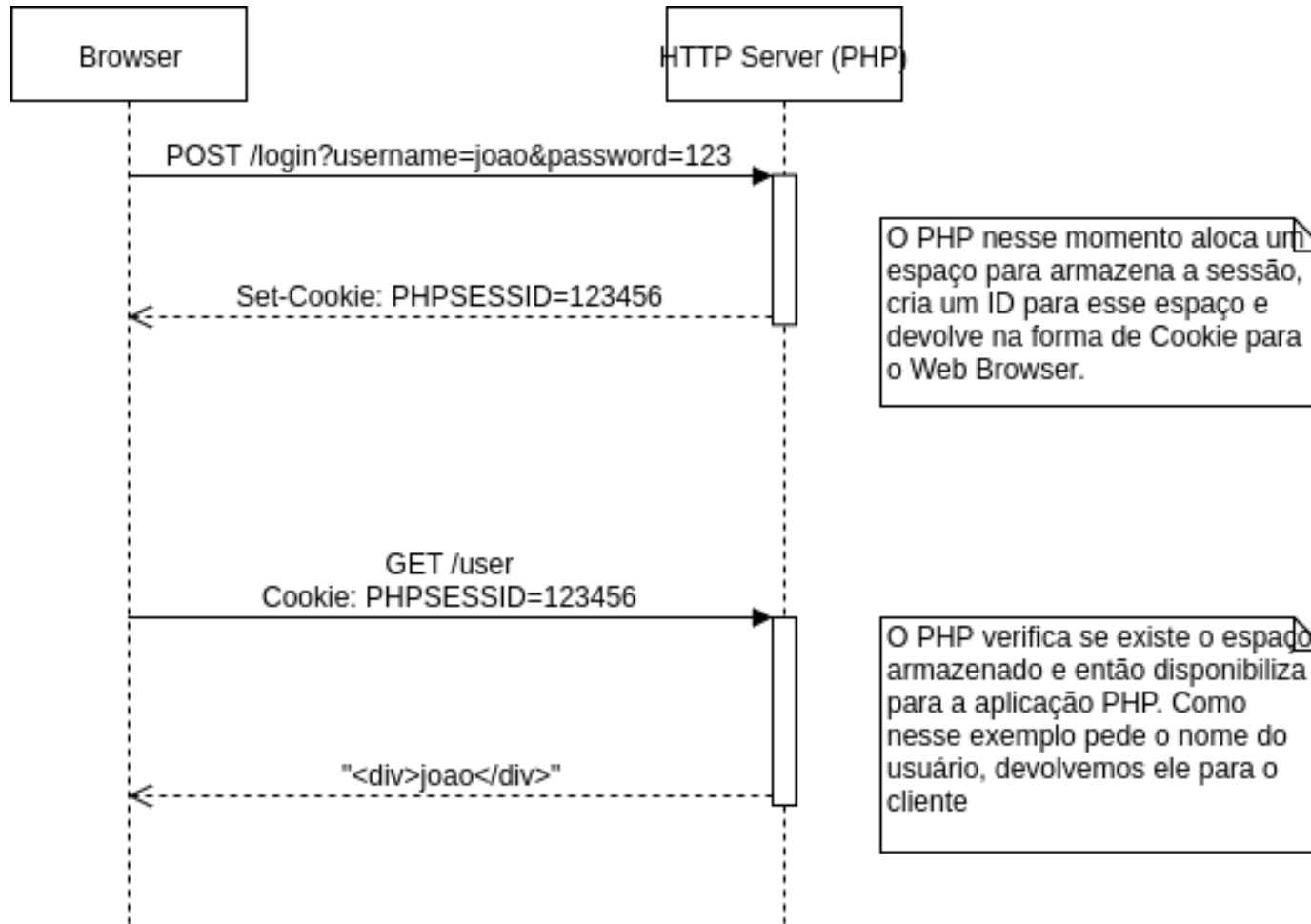
```
<?php //logout.php
session_start();

$_SESSION = array(); // 1. Limpa todas as variáveis da sessão

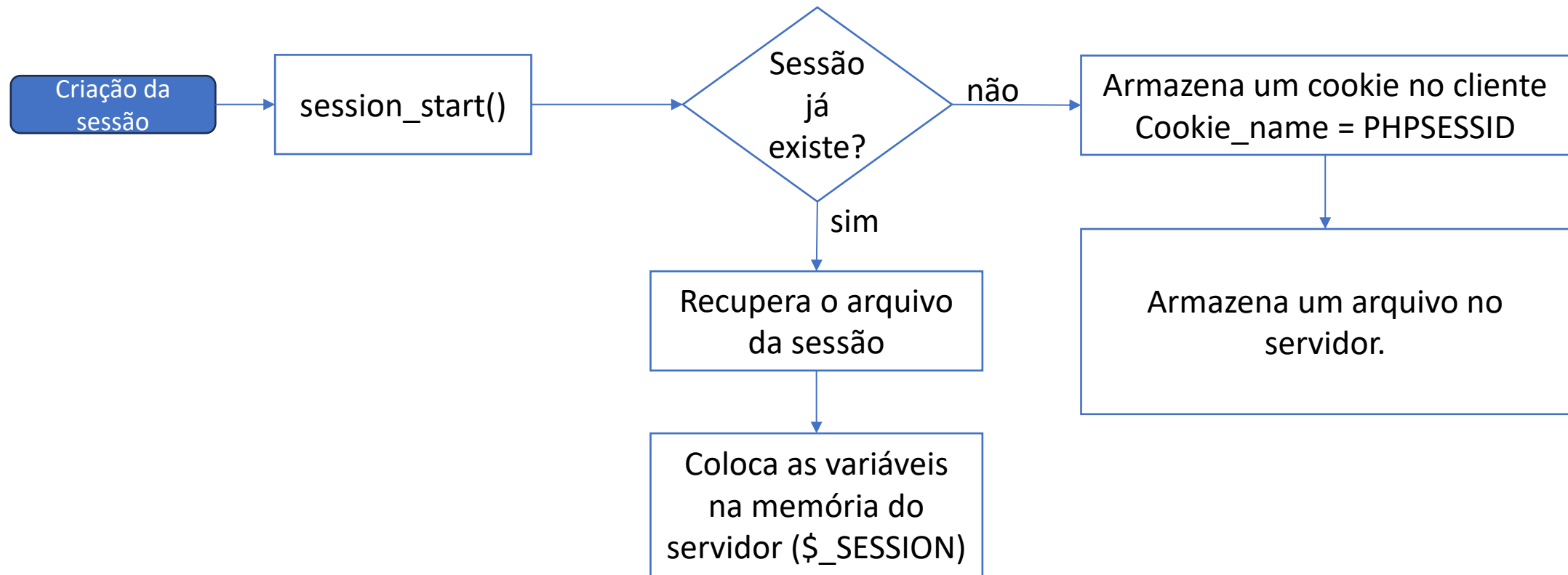
// 2. Destrói o cookie da sessão no navegador
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(session_name(), '', time() - 42000, $params["path"], $params
        ["domain"], $params["secure"], $params["httponly"]);
}
// 3. Finalmente, destrói a sessão no servidor
session_destroy();

// Redireciona para a página de login
header("Location: login.html");
exit();
?>
```

Sessão



Sessão



Sessão - exemplo

- `$_SESSION['logado'] = True;`
 - Cria ou atualiza a chave `logado` na sessão
- `unset($_SESSION['logado']);`
 - Remove a chave `logado` da sessão
- `isset($_SESSION['logado'])`
 - A função `isset()` pode ser usada para verificar se uma chave está no array.

Sessão - exemplo

- `$_SESSION = array()` ou `session_unset()`:
libera todas as variáveis atualmente registradas na sessão.
- `session_destroy()`: destrói todos os dados associados com a sessão atual.

Exercício

5. Crie um formulário de login (com usuário e senha). Envie os dados do formulário via POST para um script PHP que fará a validação dos dados no servidor (condição if).
 - a) Se a validação do usuário e senha estiver correta, coloque uma informação na sessão indicando que o usuário está logado e redirecione para uma página (home.php) com o texto “Olá, fulano!”. Troque o texto “fulano” pelo usuário logado (digitado no form).
 - b) Caso contrário, volte a requisição para a página de login informando que o usuário não foi encontrado.
6. Faça uma verificação na página home.php para não deixar acessá-la sem que se tenha um usuário logado. Inclua um botão “Sair” para deslogar o usuário (retirar os dados da sessão e redirecionar para a página de login).

Referências

- <https://www.php.net/docs.php>
- LOCKHART, J. e STURGEON, P. PHP: The Right Way. Lean Pub, 2016. Tradução disponível em <http://br.phptherightway.com>